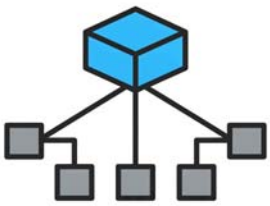
B 3.1.3 Static and Non-Static



B3.1.3 Distinguish between static and non-static variables and methods.

This section explains static vs. non-static (instance) variables and methods: their differences, usage, and scope. It covers when to use instance vs. class variables and how to apply these concepts in code.

Learning Objectives: Java

By the end of this lesson, students will be able to:

- Define static and non-static variables and methods in Java.
- Explain the difference in how static and non-static members are associated with a class and its objects.
- Describe the scope of static and non-static members within a Java program.
- Identify appropriate use cases for static and non-static members based on program requirements.
- Write simple Java code that correctly implements and utilises both static and non-static variables and methods.

Relevant ATL Skills

- **Thinking Skills:**
  - **Understanding and Applying Concepts:** Students will understand and apply the concepts of static and non-static members.
  - **Critical Thinking:** Students will analyse scenarios to determine the appropriate use of static or non-static members.
  - **Transfer:** Students will apply their understanding of scope and usage to different coding problems.
- **Communication Skills:**
  - **Explaining Concepts:** Students will be able to articulate the differences between static and non-static members.
  - **Interpreting Code:** Students will analyse code examples that demonstrate the use of static and non-static members.
- **Self-Management Skills:**

- **Organisation:** Students will organise their understanding of the concepts through activities and worksheets.

Learning Objectives: Python

By the end of this lesson, students will be able to:

- Define instance attributes/methods and class attributes/methods in Python.
- Explain the difference in how instance and class members are associated with a class and its objects in Python.
- Describe the scope of instance and class members within a Python program.
- Identify appropriate use cases for instance and class members based on program requirements in Python.
- Write simple Python code that correctly implements and utilizes both instance and class attributes/methods.

Relevant ATL Skills:

- **Thinking Skills:**
  - **Understanding and Applying Concepts:** Students will understand and apply the concepts of instance and class members.
  - **Critical Thinking:** Students will analyse scenarios to determine the appropriate use of instance or class members.
  - **Transfer:** Students will apply their understanding of scope and usage to different coding problems.
- **Communication Skills:**
  - **Explaining Concepts:** Students will be able to articulate the differences between instance and class members.
  - **Interpreting Code:** Students will analyse code examples that demonstrate the use of instance and class members.
- **Self-Management Skills:**
  - **Organisation:** Students will organise their understanding of the concepts through activities and worksheets.

Lesson Resources

Presentation

Link to presentation Java

Link to presentation Python

eBook

B 3.1.3 Static and Non-Static Java

B 3.1.3 Static and Non-Static Python

Other Resources

B 3.1.3 Worksheet Java

B 3.1.3 Worksheet Python

Lesson Plan Table (Java)

Time	Activity	Content	Covered Slide(s)
0-5 min	**Introduction & Learning Objectives**	Briefly introduce the topic and state the learning objectives for the lesson.	1
5-10 min	**Recap - Classes and Objects**	Review the fundamental concepts of classes and objects as a foundation for understanding static and non-static members.	2
10-20 min	**Non-Static Members (Variables & Methods)**	Explain non-static (instance) variables and methods, their characteristics, and how they relate to individual objects. Provide examples.	3, 4, 5, 6
20-30 min	**Static Members (Variables & Methods)**	Explain static (class) variables and methods, their characteristics, and how they belong to the class itself. Provide examples.	7, 8, 9, 10
30-35 min	**Usage and Scope Comparison**	Directly compare the usage and scope of non-static and static members.	11, 12
35-40 min	**When to Use Which?**	Provide guidelines and scenarios to help students determine when to use static versus non-static members.	13
40-45 min	**Quick Check & Questions**	Pose quick questions to check understanding and address any immediate student queries.	14
45-55 min	**Worksheet Activity (Java)**	Distribute the Java worksheet and allow students time to work on the exercises individually or in pairs. Circulate and provide support.	15, Worksheet
55-60 min	**Wrap-up & Summary**	Briefly review the key takeaways from the lesson and answer any remaining questions. Preview the next steps.	16

Lesson Plan Table (Python)

Time	Activity	Content	Covered Slide(s)
0-5 min	**Introduction & Learning Objectives**	Briefly introduce the topic and state the learning objectives for the lesson.	1
5-10 min	**Recap - Classes and Objects (Python)**	Review the fundamental concepts of classes and objects in Python as a foundation.	2
10-20 min	**Instance Attributes & Methods**	Explain instance attributes (using `__init__` and `self`) and instance methods, their characteristics, and how they relate to objects.	3, 4, 5, 6
20-30 min	**Class Attributes & Methods (Static)**	Explain class attributes (declared outside methods), class methods (`@classmethod`), and static methods (`@staticmethod`).	7, 8, 9, 10, 11
30-35 min	**Usage and Scope Comparison**	Directly compare the usage and scope of instance and class members in Python.	12, 13
35-40 min	**When to Use Which?**	Provide guidelines and scenarios to help students determine when to use instance versus class members in Python.	14
40-45 min	**Quick Check & Questions**	Pose quick questions to check understanding and address any immediate student queries.	15
45-55 min	**Worksheet Activity (Python)**	Distribute the Python worksheet and allow students time to work on the exercises individually or in pairs. Circulate and provide support.	16, Worksheet
55-60 min	**Wrap-up & Summary**	Briefly review the key takeaways from the lesson and answer any remaining questions. Preview the next steps.	17

Notes for Teacher

Potential Challenges for Students (Java & Python):

1. **Conceptual Difference:** The core challenge lies in understanding the distinction between something that belongs to the blueprint (class) versus something that belongs to an actual instance created from that blueprint (object). Analogies are crucial here. Revisit the car analogy or use other relatable examples like cookie cutters and cookies.
2. **Scope Confusion:** Students might struggle with when and how they can access static and non-static members. Emphasize that non-static members require an object to be accessed, while static members can be accessed through the class itself. Explain the error messages they might encounter if they try to access them incorrectly.
3. **When to Use Which:** Deciding whether a member should be static/class-level or non-static/instance-level requires critical thinking. Provide various scenarios and guide students through the reasoning process. Encourage them to ask: "Does this piece of data or behaviour need to be unique for each object?"
4. **Syntax (Language Specific):**
    - **Java:** Forgetting the `static` keyword for class-level members. Incorrectly trying to access non-static members from static methods without an object instance.
    - **Python:** Understanding the role of `self` in instance methods and attributes. Grasping the purpose and syntax of `@staticmethod` and `@classmethod` decorators, and when to use each.

Strategies to Overcome Challenges:

1. **Visual Aids:** Use diagrams to illustrate how static variables are stored once at the class level, while each object has its own set of non-static variables.
2. **Code Tracing:** Walk through code examples step by step, highlighting how static and non-static members are accessed and modified. Encourage students to predict the output.
3. **Real-World Examples:** Relate the concepts to real-world scenarios beyond the classroom. For example, the number of legs a dog has could be considered static for a general `Dog` class, while its name is non-static.
4. **Error Analysis:** When students encounter errors in their code, guide them in understanding the error message in relation to static and non-static access.
5. **Peer Teaching:** Encourage students to explain the concepts to each other. This can help solidify their own understanding and identify areas of confusion.
6. **Progressive Difficulty:** Start with simple examples and gradually introduce more complex scenarios in the worksheets.
7. **Reinforcement:** Revisit the concepts in future lessons when they arise naturally in other programming contexts.
8. **"Think Alouds":** When solving problems, verbalise your thought process for choosing static or non-static members.

being blocked and legal action being taken against you.

Report a problem